Week 1

Drone setup

- Wifi antenna, cube flight controller, stabilized gimbal camera, jetson orin nano, two gps receivers and other sensors
- Object detection started with opencv, now using yolo, aerial stitching getting better results with opendronemap, still offline
- Fc hardware like cube orange or pixhawk, firmware like ardupilot px4 or betaflight, handles keeping drone stable
- Jetson or pi on ubuntu runs the bigger decisions, ros helps move sensor data and commands around
- Fc and sensors feed position velocity and body rates into our code, algorithms decide next movement, mavlink commands back to fc over a link like uart

Projects

- Competition priorities, timed waypoint flights, mapping and surveying, detecting and locating targets, dropping payloads
- Research ideas, navigation without gps, localization and mapping, faster controllers, groups of drones working together
- Stitching needs to move onto the drone and happen during flight, current setup needs extra operator and takes too long

Autonomy stack

- Ardupilot or px4 for flight control, python apps for a basic setup, ros 2 or custom middleware for more complex systems
- Waypoint following is a starting point, add perception target location and mission decisions
- Reliable system needs failsafes, backup sensing, health checks and logs, industry also pushing beyond visual line of sight and swarms

Learning

- Ai useful for concepts code and papers, check details against docs and actual experience
- Build projects, set small goals and track progress, ask teammates, learn the physical drone and other subteams too

Week 2

Aerial imagery

- Uses in rescue farming environmental work policing and competitions
- Nadir looks straight down, oblique at an angle, cameras can give rgb depth or multispectral data, gimbal steadies images

Opencv

- Free open source vision library, python and c plus plus interfaces
- Feature detection, 3d reconstruction, motion and optical flow, image cleanup, video reading and writing
- Club mostly using keypoints and matching plus image manipulation, sift and orb find useful features between frames

- For stitching, extract video frames, match features, align and overlay, fewer sampled frames can reduce work

Opendronemap

- Start with photos and accurate geotags, tune lots of settings, let odm combine them into a map
- Less custom code, useful apis and web app, can also produce a 3d model
- Can be slow without careful settings, poor gps tags cause trouble, not designed around processing during flight
- More features than needed when the goal is just an orthophoto, a corrected overhead image

Next projects

- Custom opencv stitching or structure from motion running live on the drone
- Other route, break odm processing into smaller pieces and automate incremental stitching
- More possibilities, slam for mapping while locating the drone, optical flow, terrain reconstruction

Practice ideas

- Color masks, convert bgr to hsv, isolate ranges then estimate centers from pixel positions
- Or split a flight video into frames and stitch, try feature matching or built in stitcher, document progress in repo

Week 3

Detection without neural networks

- Color and brightness filters narrow down image regions, blur reduces noise, contours help pick out boundaries
- Simpleblobdetector good for basic blobs, filter by size color or shape, harder with complicated objects

Neural networks

- Layers of connected nodes, weighted inputs plus biases and activation functions
- Training changes weights and biases to lower loss, rough brain inspiration rather than exact brain model

Vision models

- Cnn kernels extract features, early layers pick up edges and textures, deeper layers combine more complex patterns
- Pooling shrinks feature data, detection predicts classes and bounding boxes
- Collect varied images, annotate then train, preprocessing and augmentation matter, sam can help label data
- Practice either blob filtering or train yolov8, use roboflow options and compare results

Week 4

Fc and onboard computer

- Fc runs fast stability loops and handles pilot inputs, jetson does detection planning and mission decisions
- Commands go to fc, telemetry comes back, ros nodes on companion computer use that feedback

Sensors and peripherals

- Imu has accelerometers and gyros, acceleration and rotation for attitude control
- Barometer estimates height from pressure, magnetometer gives heading and helps with yaw drift
- Gps gives position velocity and time, supports waypoints return home and geofencing, club rtk setup aims for centimeter accuracy
- Rgb camera for mapping and vision, gimbal steadies view, resolution field of view frame rate and interface matter
- Pilot transmitter sends stick and switch inputs over radio, receiver passes them to fc

Communication

- Uart point to point serial, transmit and receive lines, used for companion computer gps and telemetry
- Can shared network for several devices, robust error checking, useful for escs gimbals and sensors
- I2c shares data and clock lines for sensors, spi another sensor bus with clock and device selection
- Mavlink defines commands and telemetry messages, separate from the physical connection, used with serial or udp links
- Dds supports ros 2 publish subscribe communication, hybrid example uses mavlink to ground station and dds for onboard exchange

Firmware and control

- Firmware determines flight modes and features, lots of parameters to fit the airframe
- Pid, proportional reacts to current error, integral to accumulated error, derivative to change in error, tune for stable response
- Ardupilot strong for missions and mapping, betaflight for responsive racing and acro with limited autonomy
- Px4 useful for research ros 2 and simulation, protocol exercise is choose compatible fc sensors and wiring

Week 5

Nodes

- Ros 2 libraries and tools on top of an os, usually ubuntu, programs exchange typed messages in python or c plus plus
- Split jobs into camera detector planner and fc bridge, smaller nodes easier to test restart and swap
- Nodes use topics services and actions, dds handles discovery without a ros 1 style master

Topics

- Publishers send streams, subscribers listen, no reply needed, several senders or listeners possible
- Camera frames detections odometry and battery are topic examples, message layout defined in msg files
- Publishing asynchronous, queues hold recent messages, slow consumers can lose older data

Services

- Client asks server for one quick response, use for arming mode changes or resetting odometry
- Srv files define request and response, one server should handle a service name
- Use asynchronous call with completion callback, blocking inside a callback can deadlock the default executor

Actions

- Send a goal, server accepts or rejects, sends progress then final result, client can request cancellation
- Good for flying to waypoints takeoff landing or following a path, longer jobs need feedback and a way to stop
- Action file order is goal then result then feedback

Qos

- Delivery and storage rules, reliable retries losses, best effort can drop messages, useful for fresh camera data
- Depth controls number of recent messages kept, volatile gets new data, transient local can keep earlier data for late subscribers
- Settings must match compatibly, reliable subscriber cannot receive from best effort publisher, watch px4 output topics

Callbacks and practice

- Ros runs callback functions on messages timers or requests, pass function itself without calling it
- Spin processes callbacks, keep each short so other work can run, timer period controls publish rate
- Camera feeds detector then planner, bridge supplies odometry, planner arms with service and flies with action
- Work through jazzy python publisher subscriber and service client tutorials, use course docker environment when available